Lista de papers.

State Pattern

- Propósito: Dar una buena forma de que un objeto cambie su comportamiento dependiendo de su estado interno.
- La idea es generar una nueva jerarquía polimórfica de clases que representen los estados posibles del objeto, y que el objeto tenga como colaborador interno una instancia de estas. Cuando el objeto tiene que responder a un mensaje que depende del estado, le pregunta a su instancia de estado actual como responder (el libro habla de directamente fowardear la responsabilidad al estado pero en clase dijeron que no les gustaba eso, prefieren que sea un Double Dispatch y que la responsabilidad de ejecutar vuelva al objeto original). Cuando el objeto debe cambiar de estado simplemente cambia la instancia de estado que tiene guardado.
- Estructura y Colaboraciones:
  1. Un objeto base que debe responder a sus mensajes según su estado interno. Posee un colaborador interno para almacenar la instancia de su estado. Cuando recibe un mensaje, le pregunta a su estado como responderlo adecuadamente.
  2. Jerarquía polimórfica de estados. La clase padre (abstracta) define su comportamiento (recibir los pedidos del objeto base). Cada subclase define el comportamiento específico que se debe adoptar en cada caso en ese estado.
- Ventajas:
  1. Encapsula y separa el comportamiento de cada estado. Nos permite agregar nuevos estados facilmente y sin cambiar nada de lo hecho previamente. No tenemos que lidiar con molestos ifs esparcidos en todos los métodos.
  2. Permite hacer las transiciones entre estados más explicitas (crear una nueva instancia de una clase es mucho más facil de ver que tener asignaciones de flags o de booleanos por cualquier lado).
- Ejercicio donde aparece: primer iteración de MarsRover con el colaborador interno heading.

Strategy Pattern

- Propósito: definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables.
- Strategy permite variar el algoritmo independientemente los clientes que lo usan. Captura la abstraccion en una interfaz y la implementacion la hacen las clases derivadas.
- Estructura y colaboraciones:

1. Strategy, la interfaz comun de todos los algoritmos soportados.
2. ConcreteStrategy, implementacion del algoritmo usando la interfaz anterior
3. Context. Objeto que usa la interfaz, configurandose con un objeto ConcreteStrategy. Mantiene una referencia al mismo. Puede definir una interfaz para permitirle al Stategy que accede a sus datos.

- Usar cuando muchas clases relacionadas solo se diferencian en su comportamiento. Las estrategias proporcionan una forma de configurar una clase con uno de muchos comportamientos.
- Usar cuando se necesitan diferentes variantes de un algoritmo, por trade-offs en tiempo y espacio.
- Cuando un algoritmo usa datos que el cliente no deberia de saber. Usar un patron Stategy evita exponer complejos algoritmos y estructuras de datos
- Una clase define multiples comportamientos y estos comportamientos aparecen como varios ifs en sus operaciones. En lugar de de usar condicionales, moverlos a su propia clase Strategy.

Template Method Pattern

- Propósito: definir el esqueleto de un algoritmo en una operacion, delegando la implementacion de algunos pasos a sus subclases.
- Permite que sus subclases redefinan varios pasos de un algoritmo sin cambiar la estructura del algoritmo.
- Estructuras y colaboraciones:
    1. Una clase abstracta que define las operaciones primitivas que las subclases concretas definen para implementar los pasos de un algoritmo. Implementa el esqueleto del template method, el cual usa las operaciones primitivas y tambien otras operaciones ya definidas en la clase abstracta o definidas en otros objetos.
    2. Las clases concretas que implementan las operaciones primitivas de los pasos especificos del algoritmo.
- Debe usarse para implementar las partes invariantes de un algoritmo una sola vez y dejar el resto del comportamiento para que lo implementen sus subclases.

- Se puede usar cuando hay un comportamiento comun entre subclases que debe ser factorizado y localizado en una clase comun para evitar codigo repetido.
- Se puede usar para controlar la extension de sus subclases.

Facade Pattern

- Propósito: Dar una interfaz unificada para un conjunto de interfaces de un sub-sistema. Facade define una interfaz de alto nivel que hace que el subsistema sea mas facil de usar.
- Permite wrappear un subsistema complicado en una interfaz simple.
- Estructuras y colaboraciones:
    1. La fachada (Facade). Conoce cuales objetos son responsables de resolver una peticion. Delega la peticion del cliente al objeto del subsistema apropiado
    2. Las clases del subsistema que implementan su funcionalidad. No tienen ningun tipo de conocimiento acerca de la fachada (no tienen ninguna referencia a este objeto).
- Usar cuando se desea proporcionar una interfaz sencilla para un subsistema complejo. Los subsistemas suelen volverse más complejos a medida que

  evolucionan. La mayoría de los patrones, cuando se aplican, dan como resultado clases más numerosas y más pequeñas. Esto hace que el subsistema sea más reutilizable y más fácil de personalizar, pero también se vuelve más difícil de usar para los clientes que no necesitan personalizarlo. Una fachada puede proporcionar una vista predeterminada sencilla del subsistema que es suficiente para la mayoría de los clientes. Solo los clientes

  que necesiten una mayor personalización tendrán que mirar más allá de la fachada.
- Cuando quieres hacer una capa por encima de tus subsistemas (por ejemplo en la interfaz de tus libros)
- Cuando hay muchas dependencias entre clientes y la implementacion de las clases de una abstraccion.

Object Recursion

- El objetivo de este patrón es distribuir la implementación de un método de un objeto con una estructura compleja en cada parte de la estructura. De esta forma para implementar el método el objeto puede pedirle a sus partes que lo resuelvan, y estas a su vez se lo pueden pedir a sus subpartes y asi siguiendo hasta desembocar en "casos bases".
- Estructura y colaboraciones:

1. Un cliente que inicia el proceso enviandole un mensaje "handleRequest" al objeto con la estructura compleja
2. El objeto compuesto de otros, que delegue la solución de "handleRequest" enviandole este mismo mensaje a sus partes (puede implementar comportamiento extra además de delegar).
3. Las partes que componen al objeto anterior, todas deben ser polimórficas con el objeto, osea todas deben implementar "handleRequest". Estas partes pueden tambien ser objetos complejos, por eso lo recursivo.

- Ventajas: Se permite implementar las funcionalidades sin conocer de antemano como esta formada la estructura; delega correctamente las responsabilidades, cada parte se hace cargo de su trabajo; no necesita romper encapsulamiento para implementar nada.
- Desventajas: Puede ser dificil de entender, abusarse del patrón puede volver un sistema mucho más complejo de lo que debería.
- Ejercicio donde aparece: más o menos todo el tiempo en Porfolio, suele estar intrínseco en otros patrones de diseño.

Decorator Pattern

- Propósito: Agregar responsabilidades adicionales a un objeto de forma dinámica. Es más flexible que subclasificar.
- La idea es crear nuevas clases que sean polimórficas con el objeto que se quiere decorar. Estas clases tienen como colaborador interno al objeto que estan decorando y definen la nueva funcionalidad que agregan en cada método. De este modo el cliente simplemente le manda mensajes al decorator, algunos de estos el decorator solo se los va a fowardear al decoratee, y en otros va a ser donde implemente la nueva funcionalidad.
- Desde el lado del cliente es lo mismo interactuar con un objeto decorado que con uno sin decorar porque comparten interfaz. Además este polimorfismo permite crear cadenas de decorators para agregar montones de funcionalidades nuevas.
- Estructura y Colaboraciones:
    1. Una clase abstracta que define la interfaz tanto del objeto decorado como de los decorators.
    2. La clase concreta del objeto decorado que subclasifica de la del punto 1
    3. La clase abstracta que define la interfaz de los decorators y subclasifica de la del punto 1. De entrada esta clase abstracta no aporta nada más que definir el colaborador interno 'decoratee' que todos los decorators van a tener y mantener la implementación de algún método que sea común a todos los decorators.
    4. Las clases concretas de Decorator. Estas van a fowardear sus mensajes al decoratee, y pueden agregar comportamiento antes y/o despues de fowardear. Tambien pueden

definir métodos específicos con funcionalidad enteramente propia, estos serán usados por clientes que sepan que el objeto que estan usando fue decorado con este decorator para implementar dicha funcionalidad.

- Ventajas:
  1. Es mucho más flexible que la subclasificación. Podemos añadir responsabilidad a un objeto en específico en medio de la ejecución, y despues quitarsela. También podemos combinar montones de funcionalidades muy facilmente. En contraposición si hacemos subclases deberíamos hacer una por cada nueva funcionalidad, y otras más por cada posible combinación de funcionalidades, y los objetos así creados deberían mantener las funcionalidades con las que fueron creados por toda su existencia.
  2. Nos ahorra tener que definir clases llenas de comportamiento que no les pertenece realmente.
- Desventajas:
  1. Perdemos la igualdad de objetos, ya que no sabemos si el objeto que tenemos es el verdadero o un decorator.
  2. Crea sistemas complejos de aprender y de debugear.
- Se trabaja con el último decorator de la cadena de decorators (mandan mensajes). El usuario/cliente no se da cuenta que tiene decorators de por medio.

Adapter Pattern

- Propósito: Convertir la interfaz de una clase en otra interfaz nueva que es la que los clientes esperan.
- La idea es crear un nuevo objeto que sirva de traductor entre los clientes y el objeto verdadero. Los clientes van a usar este nuevo objeto adaptador tranquilamente ya que implementa la interfaz que ellos quieren, e internamente el adaptador le va a fowardear la responsabilidad a su objeto adaptado de forma correcta.
- Estructura y Colaboraciones:
  1. Una clase abstracta que define la interfaz que los clientes esperan.
  2. La clase del adaptador que subclasifica de la anterior.
  3. La clase del objeto adaptado que tiene su propia interfaz.
     - Los clientes le envían mensajes al adaptador y este los recibe y se los envía al objeto adaptado cumpliendo con su interfaz específica.
- Ventajas:
  1. Se puede reutilizar clases que hubieran sido incompatibles previamente, evitando así la repetición de código.
  2. Nos permite adaptar todos los objetos que tengan la misma interfaz que el objeto adaptado, pudiendo adaptar toda su jerarquía polimórfica con un solo tipo de adapter.

- Algo a tener en cuenta es que se pueden crear adaptadores parametrizados, es decir adaptadores que a la hora de crearlos les pasamos clossures que le indican como adaptar cada mensaje. De esta forma permitimos que cada objeto que quiera ser adaptado a la interfaz del cliente le indique al adaptador como debe ser adaptado.
- Se trabaja con el adapter (mandan mensajes). El usuario/cliente no se da cuenta que está laburando con un adapter.

Proxy Pattern

- Propósito: proveer un placeholder de otro objeto y que controle el acceso a este.
- La idea es que en vez de tener un puntero a un objeto, tener algo con más funcionalidad pero que cumpla el mismo rol de referenciar el objeto deseado. Esto nos permite por ejemplo si tenemos un objeto en la base de datos, en vez de que cada vez que queramos interactuar con este tengamos que ir a buscarlo al base de datos, podamos tener un representante local y solamente ir a buscar el objeto real cuando sea estrictamente necesario.
- El texto separa en 3 tipos de proxies:

```
- **Remote proxy**: Provee una representación local para un objeto que se encu
- **Virtual proxy**: crea objetos caros on-demand. Ejemplo, cuando abrís un do
- **Protection proxy**: controla el acceso original al objeto. Útil para tener
```

- Estructura y Colaboraciones:

    1. Una clase abstracta que define la interfaz del objeto que queremos proxiear.
    2. La clase concreta del objeto que subclasifica de la anterior
    3. La clase del proxy, que subclasifica de la del punto 1 y tiene un colaborador interno donde se guarda su objeto proxiado.
    - El cliente interactua con el proxy de forma transparente como si fuera el objeto real, y el proxy implementa en cada mensaje la funcionalidad que necesite para controlar el acceso al objeto real como más guste.

- Ventajas:

    1. El proxy provee una capa de indirección a la hora de acceder a un objeto. Esto permite realizar distintas cosas, por ejemplo esconder el hecho de que el objeto real esta en una base de datos y no en memoria, proveer optimizaciones a la hora de responder ciertos mensajes, o dar una capa de seguridad al objeto proxiado.

- Se trabaja con el proxy (mandan mensajes). El usuario/cliente no se da cuenta que es un proxy.

Composite Pattern

- Propósito: Componer objetos para obtener estructuras arboreas que representen jerarquías en nuestro modelo. Nos permite tratar objetos individuales y objetos compuestos de forma uniforme.
- La idea es hacer que tanto los objetos individuales de nuestro modelo, como los objetos compuestos (osea los que son contenedores de objetos individuales) tengan la misma interfaz. De esta forma no solo podemos agregar objetos individuales a un objeto compuesto, sino tambien otros objetos compuestos, lo que deriva en toda una estructura arborea. De esta forma el cliente puede interactuar con la estructura de la misma forma que si fuera con un objeto individual, de forma totalmente transparente.
- Estructura y Colaboraciones:
  1. Una clase abstracta que define la interfaz tanto de los objetos individuales como de los compuestos.
  2. Clases concretas de objetos individuales, estos van a representar las hojas de nuestra estructura arborea.
  3. Clases concretas de objetos compuestos, estas van a tener un colaborador interno que sea una lista de sus hijos, y va a implementar la funcionalidad de agregar y quitar hijos.
     - El cliente va a interactuar de igual forma con objetos individuales que con objetos compuestos. Si le envía un mensaje y es un objeto individual entonces el mensaje se responde directamente, si es un objeto compuesto entonces se le fowardea a cada uno de sus hijos, pudiendo implementar funcionalidad adicional antes y/o después de fowardear.
- Ventajas:
  1. Permite crear estructuras arbitrariamente complejas y que sean tratadas de igual forma que los objetos individuales.
  2. Permite agregar nuevos componentes de forma muy sencilla.
- Desventajas:
  1. Esta transparencia entre todos los objetos, tanto individuales como compuestos, hace mucho más complicado crear objetos compuestos que solo puedan tener de hijos ciertos objetos de la jerarquía y no todos.
- Ejercicio donde aparece: en Portfolio, cuando calculamos el balance de los Portfolios.

Visitor Pattern

- Propósito: Representar una operación que se realiza sobre una estructura de objetos. Nos permite definir la nueva operación sin tener que ir a redefinir las clases de los elementos de la estructura.
- La idea es definir una operaciones que se realizan sobre una estructura de objetos. Como dijimos, no queremos que esten implementados en cada uno de estos objetos como responder las operaciones, por lo que creamos un objeto separado, un visitor. Este objeto va a ser llevado por toda la estructura y con un double dispatch a cada elemento el se encarga de ejecutar la operación correspondiente. De esta forma podemos agregar nuevas operaciones muy facilmente agregando nuevos visitors.
- Estructura y Colaboraciones:
    1. Una primer clase abstracta que define la interfaz de los elementos de la estructura de objetos mencionada. A esta interfaz le vamos a agrear el método para aceptar visitors.
    2. Las clases concretas de elementos que implementan lo anterior.
    3. Otra clase abstracta que define la interfaz de los visitors. Esta interfaz debe tener un método especifico para visitar cada clase concreta de las anteriores.
    4. Las clases concretas de visitors que implementan lo anterior.
    - Para usar un visitor primero creamos su instancia y luego se la mandamos a la estructura de objetos para que la recorra. A cada nodo de la estructura se le va a enviar el mensaje de acceptVisitor con el visitor como argumento. Estos nodos van a implementar el mensaje de acceptVisitor mandandole al visitor que reciben el mensaje especifico de visitar para su clase, por ejemplo 'aVisitor visitConcreteElementA: self'. El visitor define cada uno de estos mensajes para implementar la operación correspondiente.
- Ventajas:
    1. Agregar operaciones es tan simple como agregar un nuevo visitor. La nueva funcionalidad no queda dispersa por toda la estructura.
    2. Permite mantener la esencia de los objetos visitados, no agregandoles responsabilidades que no les pertenecen.
    3. El visitor puede acumular los resultados de las operaciones de los elementos como colaborador interno. Si no usasemos un visitor tendriamos que ir pasando los resultados parciales como argumentos o guardarlo en variables con acceso global.
- Desventajas:
    1. Agregar nuevos elementos a la estructura es complicado, ya que tenemos que redefinir todos los visitors. Si ese es nuestro caso no conviene usarlo.
    2. Podría ser que para ciertas operaciones el visitor necesite información interna de los nodos que visita, poniendo en peligro el encapsulamiento de estos.
- A tener en cuenta: La forma que se recorre la estructura de objetos generalmente esta definida en el objeto que engloba toda esa estructura. Por ejemplo una lista va a hacer que

el visitor recorra la estructura iterando por cada uno de sus elementos. Esto es generalmente lo mejor pero podría ser el caso que queramos que los visitors tengan esa responsabilidad para que puedan recorrer la estructura de formas distintas y complejas según la operación que realicen. Remarco distintas y complejas, si en todos los visitors se recorre igual dejar esa responsabilidad en cada visitor sería tener código repetido.

- Ejercicio donde aparece: en Portfolio2, cuando hicimos el ReportSummary, TransferNet, etc., visitamos las estructuras y terminamos con un dispatch al visitor (que en este caso sería, por ej, ReportSummary).

Observer Pattern

- Propósito: Crear una forma eficiente de que muchos objetos puedan depender del estado interno de un objeto.
- La idea es que yo tengo un objeto que tiene cierta información o estado interno, y otros diversos objetos que dependen de esa información. Queremos que si hay un cambio en el primer objeto los segundos se actualicen automáticamente. Para evitar que los objetos esten totalmente acoplados lo que hacemos es que el primer objeto envie una notificación a todos los demás objetos que lo estan observando.
- Estructura y Colaboraciones:
  1. Una clase abstracta que define la interfaz del objeto observado y el colaborador interno en el cual se guardara los observadores. Esta interfaz debe incluir los métodos para agregar y quitar observadores y para enviarles la notifiación.
  2. Una clase abstracta que define la interfaz de los observadores. Esta interfaz debe implementar el método para recibir la notificación del objeto observado.
  3. Las clases concretas de las anteriores
  - Cuando en el objeto observado se produce un cambio, este recorre toda su colección de observadores y le envía a cada uno un mensaje de notificación del cambio. Cada observador recibe este mensaje y actualiza su propio estado de forma correspondiente. Podría darse tambien que el observador conozca al objeto observado y pueda pedirle más información al respecto del cambio.
- Ventajas:
  1. Permite desacoplar el objeto observado de sus observadores. Esto nos permite agregar nuevos observadores al objeto de forma dinámica y quitarlos sin preocupaciones, y crear nuevos tipos de observador de forma muy simple.
- Este patrón en particular tiene un montón de consideraciones para cada caso específico de uso. No creo que resumirlas tenga mucho valor para la materia, pero si quieren ver de que se trata lean la sección de implementación del patrón.
- Ejercicio donde aparece: MarsRover: ahora más rover que nunca (la segunda parte del MarsRover). Lo usamos para suscribirnos a los cambios de posición y heading del

MarsRover, hacíamos que el MarsRover le avise a sus observers "che, cambié tal cosa"; también está la solución superadora, que es usar una variable que notifique a sus observers para sacarnos el código repetido que nos quedaba.

Abstract Factory

- Propósito: Dar una interfaz para crear familias de objetos ("productos") relacionados o dependientes sin especificar sus clases concretas.
- La idea es generar una jerarquia que encapsule varias posibles "plataformas" (subclases) y la construccion de una gama de productos. Hay una sola familia de productos pero cada subclase decide que producto concreto crear.
- Estructura y colaboraciones:
    1. Una clase abstracta que define la interfaz de las operaciones para crear productos (AbstractFactory).
    2. Las clases concretas de la interfaz anterior que implementan las operaciones para crear productos (ConcretFactory).
    3. Una clase abstracta por cada tipo de producto que define sus interfaces (AbstractProduct).
    4. Las clases concretas de la clase abstracta anterior. Define un producto que puede ser creado por una concret factory.
    5. El cliente, que es el que usa la interfaz definida por el abstract factory y abstract product.
- Ventajas:
    1. Permite que un sistema sea independiente de como se crean, componen y representan sus productos.
- Usar cuando el sistema debe ser configurado con una o multiples familias de productos. Cuando una familia de productos relacionados estan diseniados para usarse juntos y se debe reforzar esa restriccion. O cuando se quiere dar una libreria de clases de productos y solo se quiere revelar sus interfaces y no su implementacion.

Builder

- Propósito: Separar la **construcción compleja de un objeto** de su **representación final**, de modo que la misma secuencia de pasos pueda construir **distintas representaciones**.

- La idea es que cuando crear un objecto no es un simple new con 2 parametros, sino una serie de pasos (configuracion, validaciones, partes opciones, etc) se usa un builder.
- Estructura y colaboraciones:
    1. Una clase abstracta que representa al Builder. Define una interfaz para crear las partes de un objeto complejo.
    2. La clase concreta del Builder (ConcreteBuilder) que se encarga de construir y ensamblar las partes de un producto implementando la interfaz de su clase abstracta.
    3. Un director, que es el que construye un objeto complejo usando la interfaz del builder.
    4. El Producto. Representa el objeto complejo que se esta construyendo. El ConcreteBuilder lo construye y define el proceso de ensamblado. Imcluye las clases que definen las partes constituyentes del producto y las interfaces para ensamblar dichas partes en un resultado final.
- Ventajas:
    1. Permite que el algoritmo para crear un objeto complejo sea independiente de las partes que lo conforman y de como se ensamblan.
    2. Poder **reutilizar la lógica de construcción** para múltiples representaciones (p.ej. JSON, HTML, PDF).

Factory Method

- Propósito: Definir una interfaz para crear un objeto, pero dejar que las subclases decidan qué clase concreta instanciar.
- Es decir: mover new a un método especializado para que la decisión de qué crear se haga por polimorfismo.
- Estructura y colaboraciones:
    1. El producto. Define la interfaz de los objetos que el factory method crea.
    2. El concrete product que es la implementacion de la interfaz anterior.
    3. El Creator, que declara el factory method y devuelve el producto. El Creator tambien puede definir implementaciones por defecto del factory method y devolver concret products por defecto.

4. El ConcreteCreator, que overraidea el factory method definido por el Creator y devuelve una instancia del Concret Product.

- Usar cuando una clase quiere que sus subclases especifiquen los objetos que deben crear.
- Cuando la clase no puede saber el tipo de objeto exacto que debe crear.
- Cuando se tiene una clase que no hace todo ella misma, sino que delegan parte del trabajo a otras clases (helpers), y ese helper no es unico sino que hay varias implementaciones posibles y quieres que la decision de que helper usar este en un solo lugar y que el resto del sistema solo conozca la interfaz Helper.

Pattern Abuser

- Este texto es mayormente una historia, una fábula si se quiere para enseñarnos una moraleja.
- Cuenta sobre una persona que solamente programaba enfocandose en hacer las cosas funcionar, y que descubre los patrones de diseño.
- Esta persona se obsesiona con los patrones y los considera la única forma correcta de programar, buscando aplicarlos a cada situación posible.
- Llega a tal punto de creencia en los patrones que comienza a medir la calidad del código en términos de cuantos patrones aplica, y se vuelve consultor de empresas que lo contrataban para que revisara su código.
- Se nos explica el nivel de delirio mostrandonos que la persona desarrollo "un patrón" para aplicar patrones, que consistía en aplicar sin ningún tipo de consideración 15 patrones distintos.
- Finalmente el protagonista se confiesa y nos cuenta que no sabe si aplicar estos patrones en realidad no estaba empeorando el código inicial, y que solamente se estaba abusando de los patrones sin saber si era lo correcto usarlos.
- La moraleja es que tenemos que diseñar con cabeza, no aplicar los patrones a diestra y siniestra. Si lo hacemos vamos a terminar con sistemas peores, más complicados de entender y que modelan mucho peor la realidad.
- Termina pidiéndole ayuda al lector.

Modern Software Engineering

- La ingeniería del software es la aplicación de un enfoque empírico y científico para conseguir soluciones eficientes y económicas para problemas prácticos del software.
- Para ser buenos ingenieros tenemos que ser expertos en aprender. El verdadero progreso y mejora en nuestros desarrollos se hace cuando aprendemos de ellos y los mejoramos y

no solo cuando funcionan. Puntos claves son:

1. Trabajar iterativamente
2. Tener feedback rápido y de calidad
3. Trabajar incrementalmente
4. Ser experimental (si no probamos cosas nuevas no aprendemos)
5. Ser empírico (si no probamos con la realidad solo nos quedamos con nuestras propias ideas erróneas1

- Tambien tenemos que ser expertos en manejar la complejidad de nuestros proyectos. Si no aplicamos técnicas para que nuestro problema complejo que queremos resolver se pueda reducir a algo más simple, partirlo en partes y demás e incluso para que se puedan organizar distintos grupos de trabajo que trabajen en el proyecto, nuestro software resultante va a ser muy pobre. Puntos claves son:
  1. Modularidad
  2. Cohesión
  3. Separación de responsabilidades
  4. Abstracción / Information Hiding
  5. Acoplamiento
- La idea de aplicar un enfoque científico a la ingeniería del software es poder tener pensamiento crítico y validar verdaderamente si lo que hacemos esta bueno o no, sin importar si es un práctica ya muy establecida, evitando ser dogmáticos.
- La ingeniería del software no es como "construir puentes", no deberíamos aplicar un pensamiento de una industria de producción como esa al software.
- Nuestra visión de nuestra profesión tendría que ser una de ingeniería en diseño.
- Tenemos que sacar provecho de que somos el único sector en donde podemos testear nuestro producto sin que costruirlo sea un esfuerzo, y poder aprender de ese testeo (TDD are you here?).
- Un problema que tenemos es que es muy dificil medir cuando nuestras formas de trabajar son buenas o no.
- Para esto el libro habla sobre la forma de medirlo que se desarrollo analizando DevOps, que se basa en medir la "estabilidad" y el "rendimiento". La estabilidad se mide según el ratio en el cual introducir un cambia produce errores, y el tiempo que se tarda en corregir los errores en el software. El rendimiento se mide en el tiempo que lleva cambiar el código para que tenga nueva funcionalidad y que tan frecuentemente se llevan a cabo los cambios (la idea es que nos indica que el desarrollo esta teniendo mucho feedback y por lo tanto se aprende más y es mejor).

Modern Software Engineering, versión alternativa: basada en las diapositivas de Sebastián

CAPÍTULO 1: ¿QUÉ ES LA INGENIERÍA DE SOFTWARE?

- **La ingeniería de software como proceso de aprendizaje**: El desarrollo de software es un proceso continuo de descubrimiento y exploración, lo que exige que los ingenieros se conviertan en expertos en aprender y adaptarse constantemente.

```
- **Un enfoque empírico**: La ingeniería de software aplica un enfoque científ
```

- **Crisis del software**: En la década de 1960, se percibió una brecha considerable entre el progreso *rápido* del *hardware* y el más *lento* del *software*, conocida como la crisis del software. Fred Brooks destacó que no hay soluciones mágicas que prometan mejoras significativas en productividad, confiabilidad o simplicidad a corto plazo.

CAPÍTULO 2: ¿QUÉ ES LA INGENIERÍA?

- **La producción no es nuestro mayor desafío**:
  - **Producción en otros campos**: La producción en masa de objetos físicos, como automóviles o teléfonos, es costosa y compleja. En software, sin embargo, la "producción" consiste en compilar el código, un proceso comparativamente simple.
    - **Mentalidad de producción**: La industria del software ha heredado enfoques como el modelo Waterfall, diseñado para líneas de producción. Sin embargo, el desarrollo de software se beneficia más de procesos que fomenten el descubrimiento, el aprendizaje y la experimentación.
    - **Ingeniería de diseño, no de producción**:
    - **Diseño frente a producción**: En el caso de un puente físico, los desafíos no solo están en el diseño, sino también en los costos y dificultades de producción. En software, el diseño es central, pero iterar y ajustar modelos es mucho más accesible.
    - **Ventajas del software**: A diferencia de los productos físicos, los modelos de software son también su resultado final, lo que permite iteraciones rápidas y económicas.
    - **La ingeniería como matemática**:
    - **Métodos formales**: Estos ofrecen validación matemática del software, pero su complejidad limita su adopción en contextos prácticos y dinámicos.
      - **Pragmatismo y empirismo**: En lugar de depender solo de la teoría, se adopta un enfoque experimental y empírico, similar a la validación en

ingeniería aeroespacial. El software, aunque más flexible, debe equilibrar formalidad y adaptabilidad en dominios complejos o concurrentes.

- **Definición operativa de ingeniería**: La ingeniería aplica un enfoque empírico y científico para resolver problemas prácticos de manera eficiente y económica. Cada palabra importa: es ciencia aplicada, orientada a la práctica y limitada por factores económicos.

CAPÍTULO 3: FUNDAMENTOS DE UN ENFOQUE DE INGENIERÍA

- **¿Es una industria cambiante?**: Aunque la industria celebra nuevos avances tecnológicos, no todos generan cambios significativos en cómo se desarrolla software. La mayor parte del progreso ha estado relacionado con niveles más altos de abstracción, como el paso de ensamblador a lenguajes de alto nivel y de paradigmas procedurales a orientados a objetos.

- **La importancia de medir**:

  - **Problemas con las métricas tradicionales**: Muchas medidas como velocity, líneas de código o cobertura de tests son irrelevantes o incluso contraproducentes.

```
- *(Métrica)* **Stability**: Rastrea la tasa de fallos de cambios y el ti
- *(Métrica)* **Throughtput**: Mide la eficiencia (lead time) y la frecue
```

    - **Aplicación de estabilidad y rendimiento**:
      - **Medición del impacto**: Estas métricas permiten evaluar cambios en procesos, tecnología o cultura. Por ejemplo, añadir un Change Approval Board puede ralentizar el proceso sin mejorar la estabilidad, evidenciando cómo decisiones mal fundamentadas pueden ser contraproducentes.
      - **Pruebas empíricas**: Al aplicar estas métricas, podemos tomar decisiones informadas sobre adopción de tecnologías o modificaciones organizacionales, priorizando mejoras en rendimiento y estabilidad.

- **Los Fundamentos de una disciplina de Ingeniería de Software**:

```
- **Convertirnos en expertos en aprender**: Debemos reconocer y aceptar que nu
- **Convertirnos en expertos en manejar la complejidad**: Creamos sistemas que
- **Cinco comportamientos clave en esta categoría**:
      - Trabajar de manera iterativa.
      - Utilizar feedback rápido y de alta calidad.
      - Trabajar de forma incremental.
      - Ser experimentales.
      - Ser empíricos.
- **Cinco ideas clave para manejar la complejidad**:
      - Modularidad.
      - Cohesión.
      - Separación de responsabilidades.
      - Encapsulamiento/abstracción.
      - Acoplamiento.
```

Respuesta sobre Computer Science de Alan Kay en Quora

- Alan Kay propone que en el pasado la ciencia de la computación era algo mucho menos popular que ahora, lo que hacía que la gente que se dedicaba a esto lo hiciera en busca de verdaderamente impulsar la nueva ciencia hacia delante y no buscar trabajo.
- Reclama de que parte de la culpa la tienen las universidades que cambiaron de modelo para ser más un negocio y poder subirse al boom de popularidad de la computación sin pensar en la calidad educativa.
- Recalca que fue shockeado cuando la universidad de Stanford determinó el uso de Java como primer lenguaje de programación para los estudiantes entrantes. Java nos da abstracciones que nos hace olvidarnos, por ejemplo, del manejo de la memoria.
- Uno de sus reclamos es que no se enseña la historia de la ciencia de la computación, ni quienes la hicieron, ni siquiera una buena definición de ciencia. Esto indica, según él, una falta de perspectiva y curiosidad intelectual.